



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto AnzenCore

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Perez Peralta, Fabrizio Salvador Elias (2023077476)

Tacna – Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
2.0	FSEPP GAPP	PCQ	PCQ	27/06/2026	Versión FINAL

**Sistema *AnzenCore* – Analizador de Vulnerabilidades
Móviles
Documento de Arquitectura de Software**

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
2.0	GFAA y FPP	PCQ	PCQ	27/06/2026	Versión FINAL

INDICE GENERAL

Contenido

1.	INTRODUCCIÓN	5
1.1.	Propósito (Diagrama 4+1).....	5
1.2.	Alcance.....	5
1.3.	Definición, siglas y abreviaturas	6
1.4.	Organización del documento	7
2.	OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS	7
2.1.1.	Requerimientos Funcionales	9
2.1.2.	Requerimientos No Funcionales – Atributos de Calidad	11
3.	REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA	13
3.1.	Vista de Caso de uso	13
3.1.1.	Diagramas de Casos de uso	13
3.2.	Vista Lógica.....	15
3.2.1.	Diagrama de Subsistemas (paquetes).....	15
3.2.2.	Diagrama de Secuencia (vista de diseño).....	16
3.2.3.	Diagrama de Colaboración (vista de diseño).....	16
3.2.4.	Diagrama de Objetos.....	16
3.2.5.	Diagrama de Clases	17
3.2.6.	Diagrama de Base de datos (relacional o no relacional).....	18
3.3.	Vista de Implementación (vista de desarrollo)	19
3.3.1.	Diagrama de arquitectura software (paquetes)	20
3.3.2.	Diagrama de arquitectura del sistema (Diagrama de componentes)	20
3.4.	Vista de procesos	21
3.4.1.	Diagrama de Procesos del sistema (diagrama de actividad).....	21
3.5.	Vista de Despliegue (vista física).....	24
3.5.1.	Diagrama de despliegue	24
4.	ATRIBUTOS DE CALIDAD DEL SOFTWARE	25
	Escenario de Funcionalidad	25
	Escenario de Usabilidad	26
	Escenario de confiabilidad.....	26
	Escenario de rendimiento.....	27

Escenario de mantenibilidad27

Otros Escenarios27

1. INTRODUCCIÓN

1.1. Propósito (Diagrama 4+1)

El presente documento describe la arquitectura de software del sistema AnzenCore, una plataforma de auditoría de seguridad móvil que aplica ingeniería inversa sobre APKs (motor Python: zipfile + struct + re), con API backend en FastAPI, un dashboard web en Streamlit y un motor de análisis de APKs en Python. El propósito es presentar una visión global de las decisiones arquitectónicas tomadas para satisfacer los requerimientos funcionales y no funcionales especificados en el Documento de Especificación de Requerimientos de Software (ERS), priorizando la separación de responsabilidades bajo el patrón MVC, la seguridad en el tránsito y procesamiento de los archivos APK, y la escalabilidad del servicio en un entorno cloud (Azure Container Apps). Las vistas se documentan siguiendo el modelo 4+1 (Casos de Uso, Lógica, Implementación, Procesos y Despliegue), priorizando la eficiencia de procesamiento del análisis estático sobre la portabilidad a otras plataformas distintas de Android.

1.2. Alcance

El presente documento se centra en el desarrollo de la vista lógica y de implementación del sistema AnzenCore, dado que constituyen los aspectos arquitectónicamente más significativos: el motor de ingeniería inversa (ApkAnalyzer + detectores) como subsistema core y la separación

MVC entre la API (FastAPI), el dashboard (Streamlit) y la base de datos (Supabase/PostgreSQL), así como el motor de ingeniería inversa como núcleo del análisis de APKs. Se incluyen también la vista de casos de uso y la vista de despliegue sobre Azure Container Apps. Se omiten o se tratan de forma resumida los aspectos de la vista de procesos que no aporten valor arquitectónico adicional, dado que el sistema no maneja procesos concurrentes pesados más allá del análisis asíncrono de APKs.

1.3. Definición, siglas y abreviaturas

El presente documento describe la arquitectura de software del sistema AnzenCore, una plataforma de auditoría de seguridad móvil que aplica ingeniería inversa sobre APKs (motor Python: zipfile + struct + re), con API backend en FastAPI, un dashboard web en Streamlit y un motor de análisis de APKs en Python. El propósito es presentar una visión global de las decisiones arquitectónicas tomadas para satisfacer los requerimientos funcionales y no funcionales especificados en el Documento de Especificación de Requerimientos de Software (ERS), priorizando la separación de responsabilidades bajo el patrón MVC, la seguridad en el tránsito y procesamiento de los archivos APK, y la escalabilidad del servicio en un entorno cloud (Azure Container Apps). Las vistas se documentan siguiendo el modelo 4+1 (Casos de Uso, Lógica, Implementación, Procesos y Despliegue), priorizando la eficiencia de procesamiento del análisis estático sobre la portabilidad a otras plataformas distintas de Android.

1.4. Organización del documento

El documento se organiza de la siguiente manera: la sección 1 presenta la introducción, propósito y alcance del documento. La sección 2 establece los objetivos y restricciones arquitectónicas, priorizando los requerimientos funcionales y no funcionales del ERS. La sección 3 desarrolla la representación de la arquitectura del sistema mediante el modelo de vistas 4+1 (casos de uso, lógica, implementación, procesos y despliegue). Finalmente, la sección 4 describe los atributos de calidad del software (escenarios de funcionalidad, usabilidad, confiabilidad, rendimiento y mantenibilidad) que sustentan las decisiones de diseño tomadas.

2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS

A continuación, se establecen las prioridades de los requerimientos funcionales y no funcionales extraídos del Documento de Especificación de Requerimientos de Software (ERS) de AnzenCore, así como las restricciones técnicas y de plazo que condicionan las decisiones arquitectónicas del proyecto.

2.1. Priorización de requerimientos

Se despliegan los requerimientos funcionales y no funcionales priorizados, lo cual define el orden de implementación del sistema:

<i>ID</i>	<i>Descripcion</i>	<i>Prioridad</i>
RF-04/05	Análisis server-side de APKs subidas: permisos peligrosos, secretos hardcodeados, vulnerabilidades, código ofuscado y persistencia segura de hallazgos	Alta
RF-06/07	Cálculo de severidad máxima y visualización de resultados en el dashboard web	Alta
RF-08	Motor IR — Desofuscación y Detección: extracción DEX, desofuscación de strings/clases ProGuard/R8, ejecución de 8+ detectores de vulnerabilidades OWASP Mobile Top 10 sobre código desofuscado	Alta
RF-08/12	Motor IR y Agente Android: motor de ingeniería inversa stateless + agente Kivy (API 26+) para escaneo local del dispositivo (permisos, certificados)	Alta
RNF-01	Cifrado TLS 1.2+ en todos los reportes de análisis transmitidos	Alta

RNF-03	Tiempo de análisis del dispositivo inferior a 60 segundos	Alta
RNF-07	Cumplimiento estricto del patrón MVC en Python (FastAPI + Streamlit)	Alta
RF-09	Dashboard — Exportación PDF de reportes de análisis IR con hallazgos, evidencias extraídas, CWE, OWASP y recomendaciones de mitigación	Media
RF-13	Usuarios conectados en tiempo real mediante ping periódico	Media
RNF-08	Escalabilidad para soportar hasta 10 000 usuarios concurrentes	Media

2.1.1. Requerimientos Funcionales

Se prioriza la totalidad de los requerimientos funcionales identificados en el ERS de AnzenCore:

<i>ID</i>	<i>Descripcion</i>	<i>Prioridad</i>
RF-01	Registro de usuarios con nombre de usuario y contraseña, validando unicidad	Alta

RF-02	Inicio y cierre de sesión seguro mediante token de sesión	Alta
RF-03	Descarga de la APK de análisis precompilada desde la plataforma web	Alta
RF-04	Análisis server-side de la APK subida: permisos peligrosos, secretos hardcodeados, código ofuscado, vulnerabilidades y metadatos	Alta
RF-05	Recepción de la APK por HTTPS y persistencia de hallazgos sin almacenar datos personales	Alta
RF-06	Cálculo de severidad máxima y conteo de hallazgos por escaneo	Alta
RF-07	Visualización del dashboard con score, hallazgos clasificados y exportación de reporte PDF	Alta
RF-08	El motor IR aplica desofuscación sobre strings DEX y ejecuta detectores de vulnerabilidades (8+ tipos) sobre el código desofuscado. Hallazgos persistidos en apk_findings con evidencia, CWE y OWASP.	Alta

RF-09	Exportación de reporte PDF con hallazgos del análisis IR, evidencias, severidad OWASP y recomendaciones de mitigación (plans Académico+)	Media
RF-10	Historial paginado de escaneos APK por usuario con comparativa entre versiones de una misma aplicación (por package name)	Alta
RF-11	Gamificación: micro-lecciones por vulnerabilidad detectada, ranking semanal de usuarios y comunidad en línea (usuarios activos, ping 30s)	Alta
RF-12	Historial de escaneos APK propios del usuario (fecha, tamaño, estado)	Alta
RF-13	Visualización de usuarios conectados en tiempo real	Media

2.1.2. Requerimientos No Funcionales – Atributos de Calidad

Se prioriza la totalidad de los requerimientos funcionales identificados en el ERS de AnzenCore:

<i>ID</i>	<i>Descripcion</i>	<i>Prioridad</i>
RNF-01	Cifrado TLS 1.2+ en la transmisión de todos los reportes de análisis	Alta
RNF-02	Prohibición de lectura/almacenamiento de contactos, mensajes o fotos por la APK	Alta
RNF-03	Análisis del dispositivo completado en menos de 60 segundos	Alta
RNF-04	Disponibilidad de la plataforma web del 99% (24/7)	Alta
RNF-05	Dashboard responsivo y usable desde pantallas de 320px	Media
RNF-06	Compatibilidad de la APK con Android 8.0 (API 26) o superior	Alta
RNF-07	Cumplimiento estricto del patrón MVC en Python (FastAPI + Streamlit)	Alta
RNF-08	Escalabilidad para soportar hasta 10 000 usuarios concurrentes	Media
RNF-09	Portabilidad de la plataforma web en Chrome, Firefox, Safari y Edge	Media

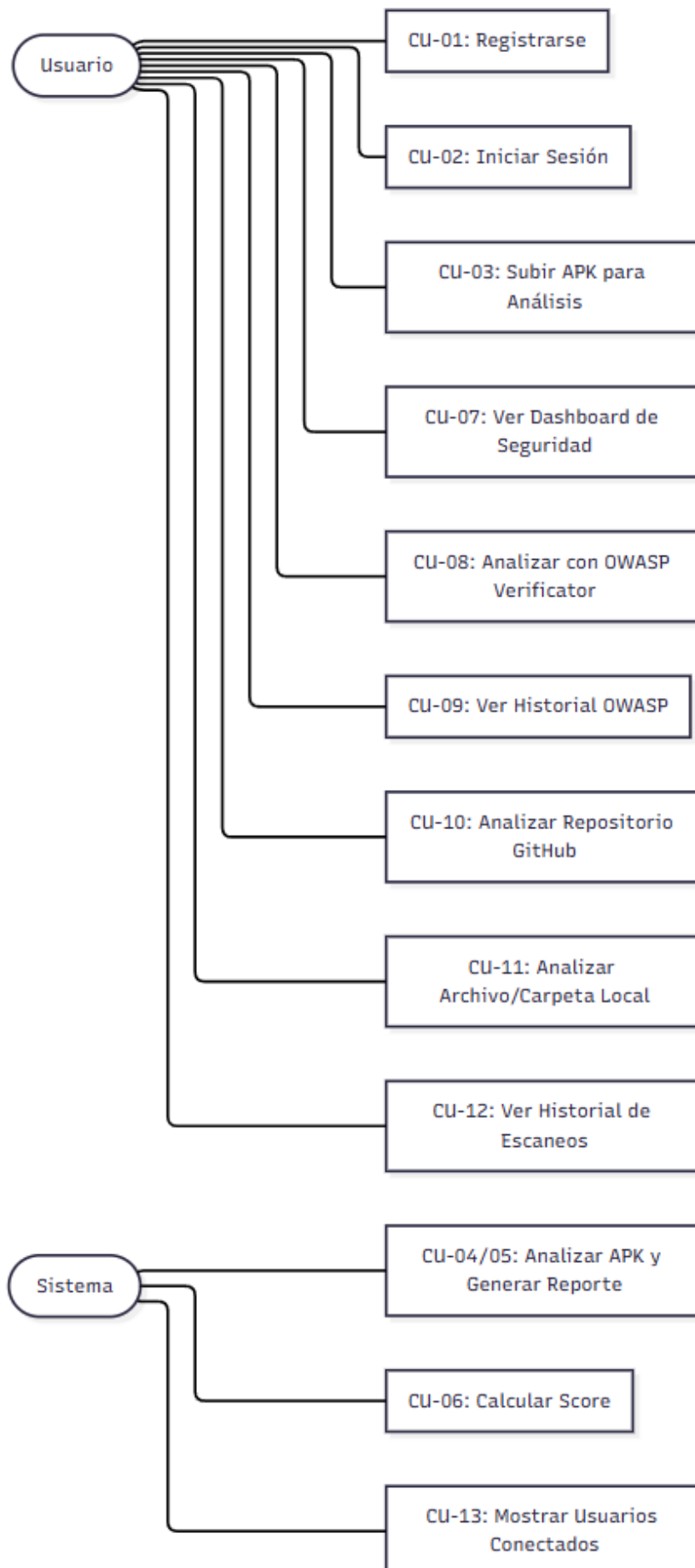
2.2. Restricciones

El proyecto se desarrolla en un entorno académico, lo que impone restricciones de tiempo (un semestre académico) y de equipo (dos integrantes). A nivel técnico, el sistema debe implementarse estrictamente bajo el patrón MVC en Python, usando FastAPI para el backend y Streamlit para el dashboard, sin posibilidad de cambiar de stack tecnológico durante el desarrollo. El almacenamiento de datos debe realizarse en Supabase (PostgreSQL) por su tier gratuito y facilidad de integración. El despliegue debe realizarse en Azure Container Apps (créditos estudiantiles Microsoft), lo que permite un contenedor de 0.5 vCPU/1 GB RAM sin costo durante el desarrollo. Por motivos de privacidad, el sistema no puede almacenar el contenido binario de la APK ni datos personales del usuario, solo los hallazgos y metadatos del análisis. Finalmente, el motor IR ejecuta el análisis de forma stateless sin almacenar el binario APK, garantizando la privacidad del código del usuario.

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

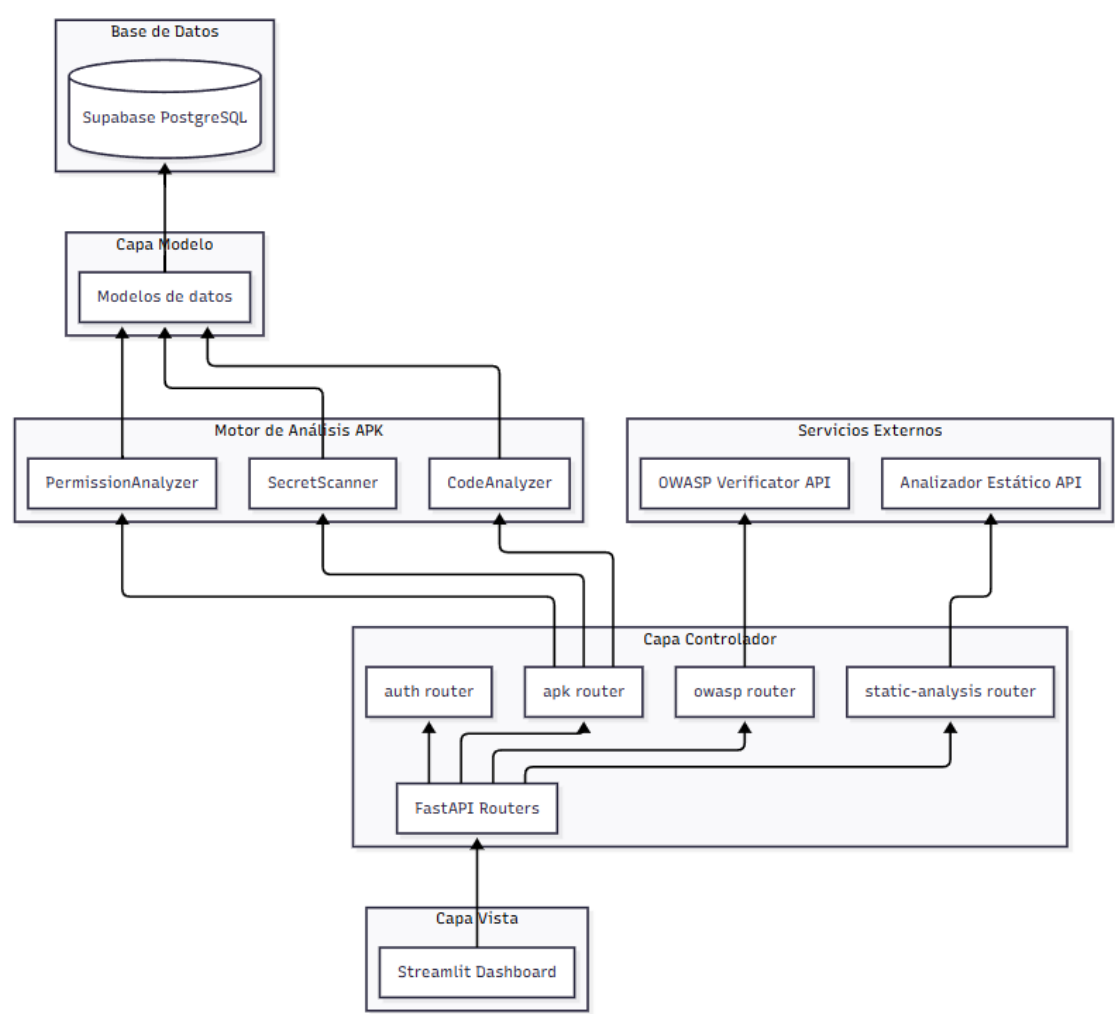
3.1. Vista de Caso de uso

3.1.1. Diagramas de Casos de uso

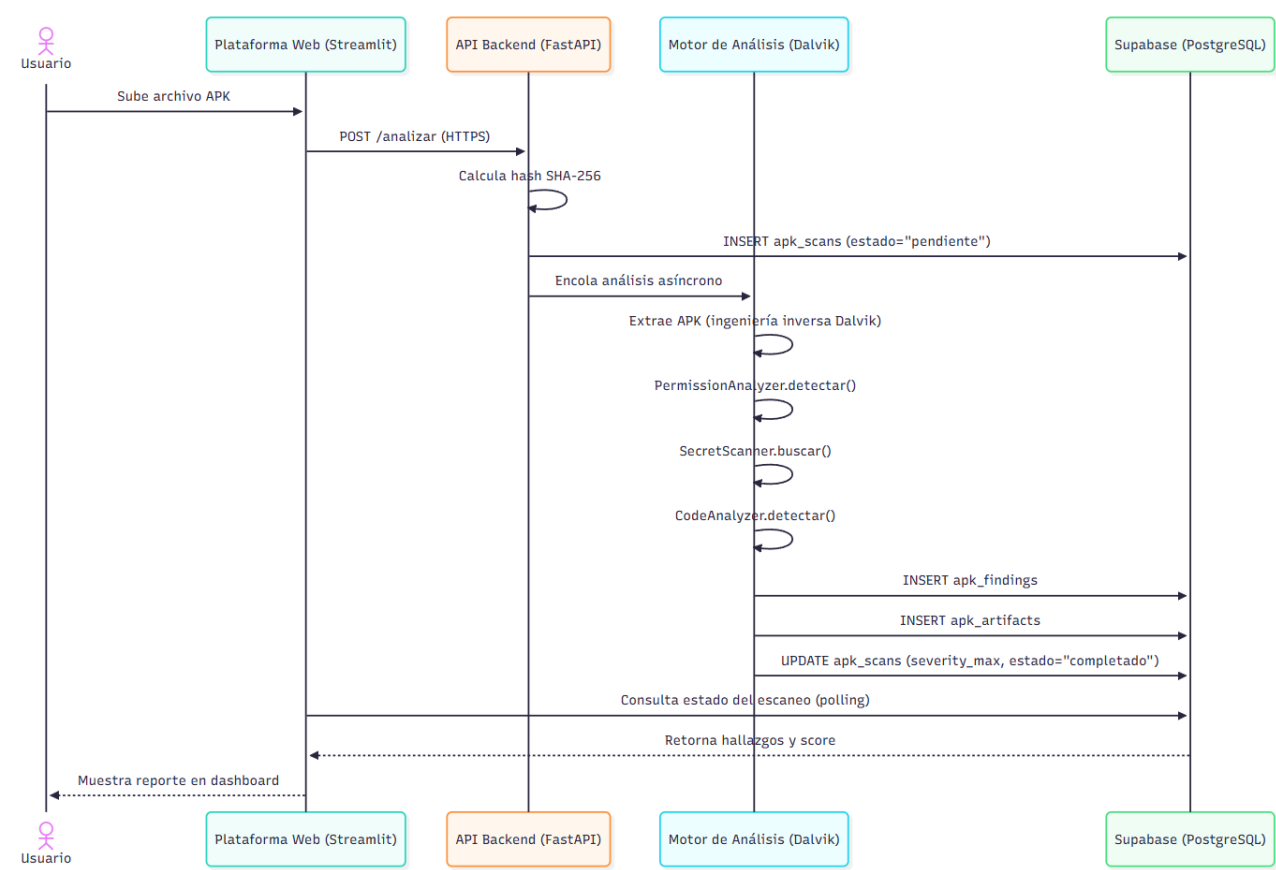


3.2. Vista Lógica

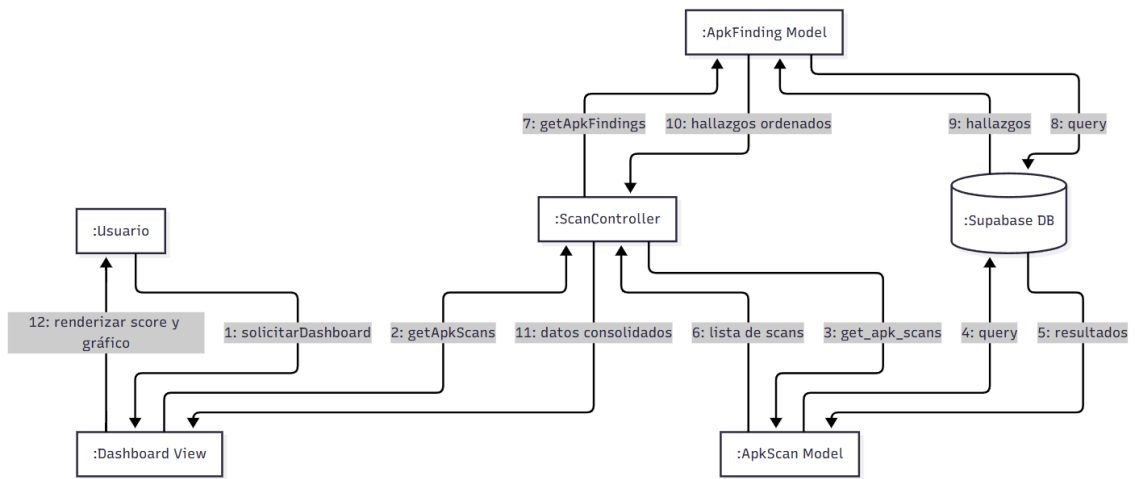
3.2.1. Diagrama de Subsistemas (paquetes)



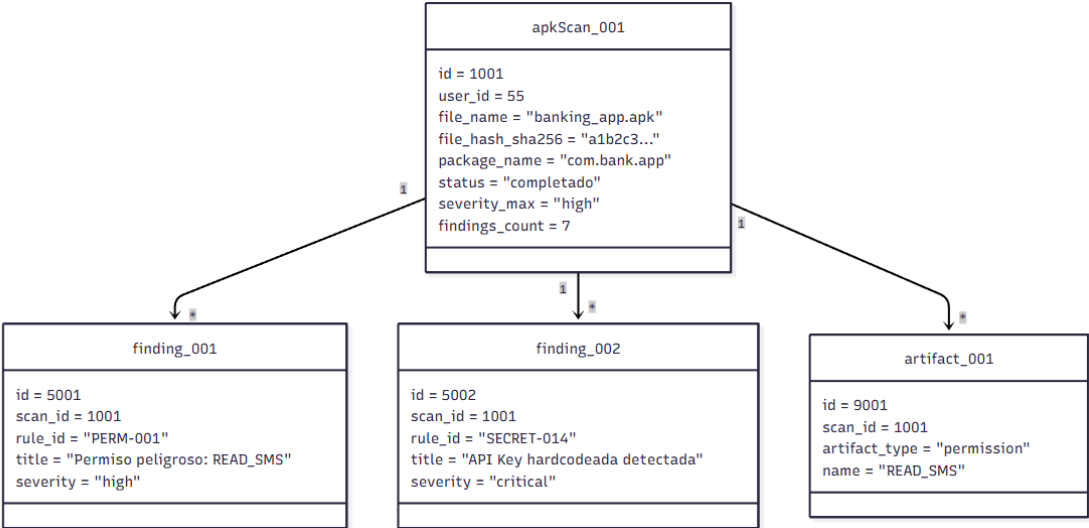
3.2.2. Diagrama de Secuencia (vista de diseño)



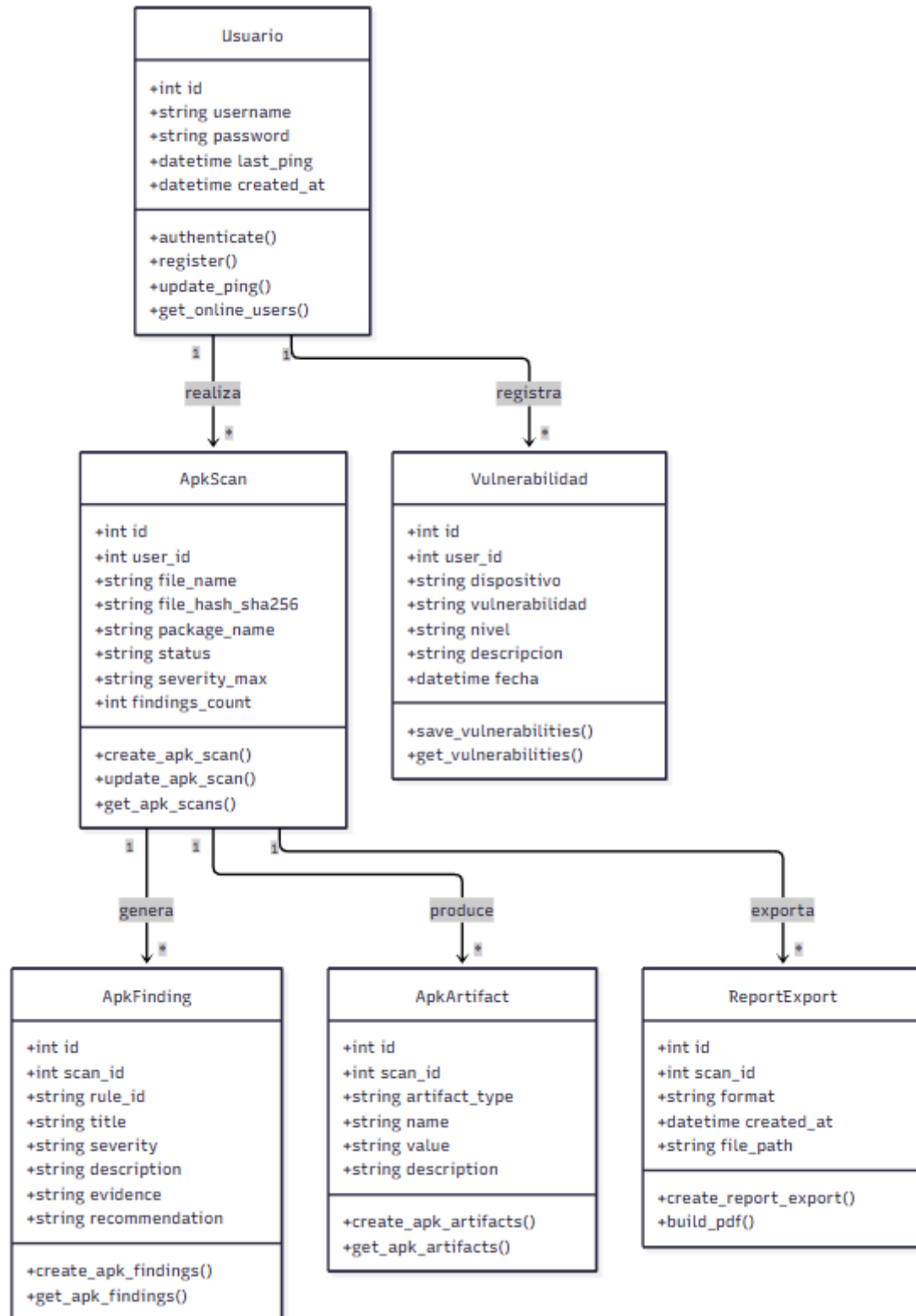
3.2.3. Diagrama de Colaboración (vista de diseño)



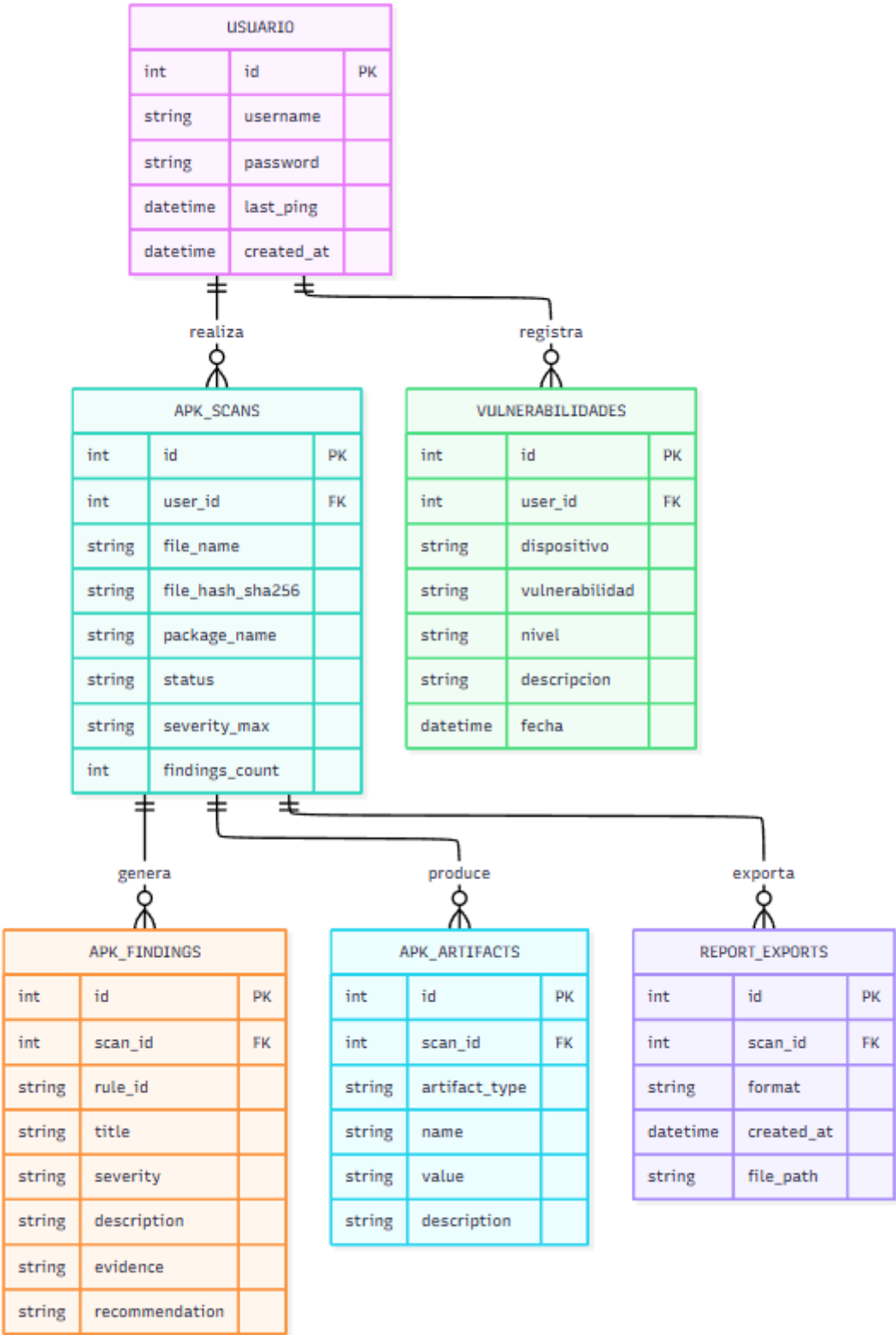
3.2.4. Diagrama de Objetos



3.2.5. Diagrama de Clases

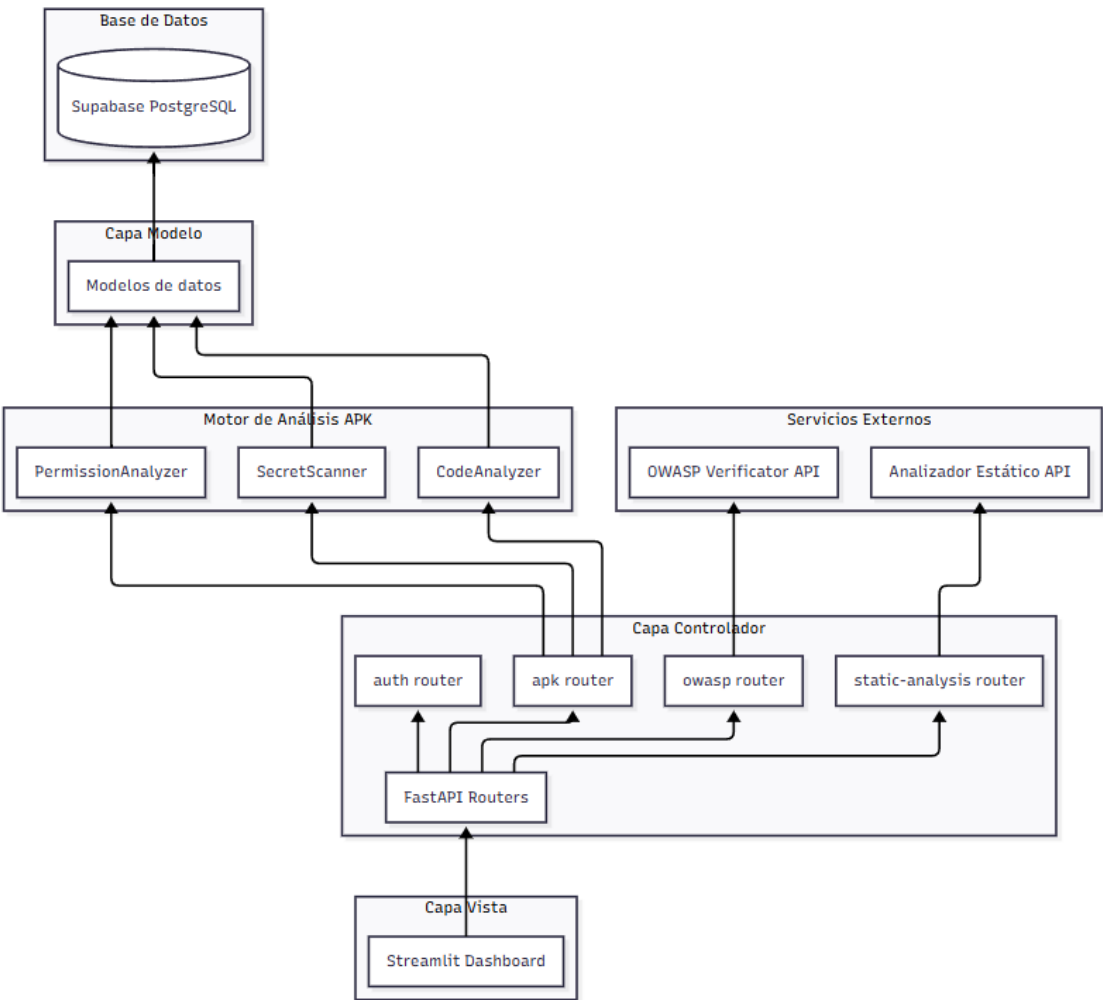


3.2.6. Diagrama de Base de datos (relacional o no relacional)

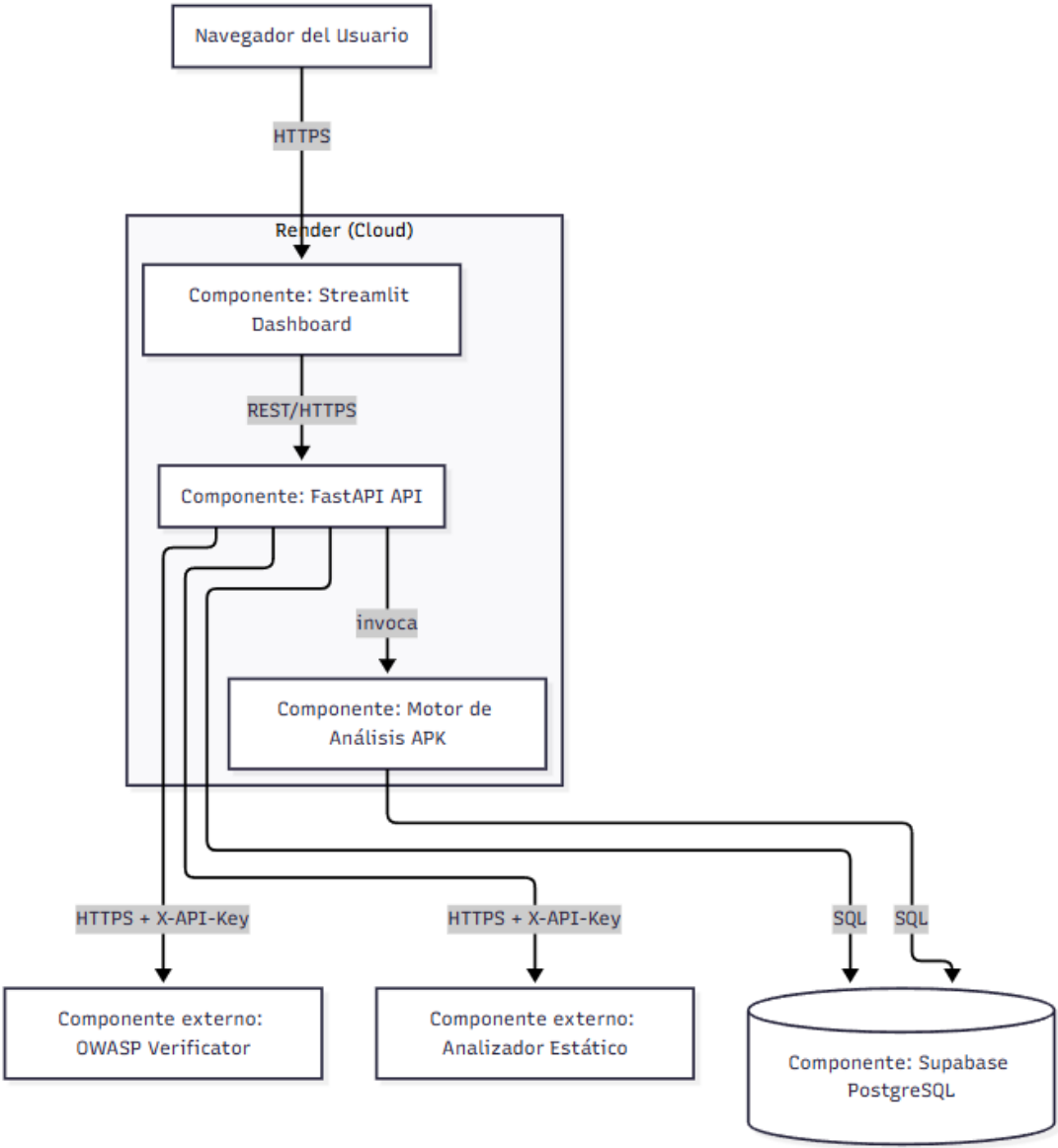


3.3. Vista de Implementación (vista de desarrollo)

3.3.1. Diagrama de arquitectura software (paquetes)

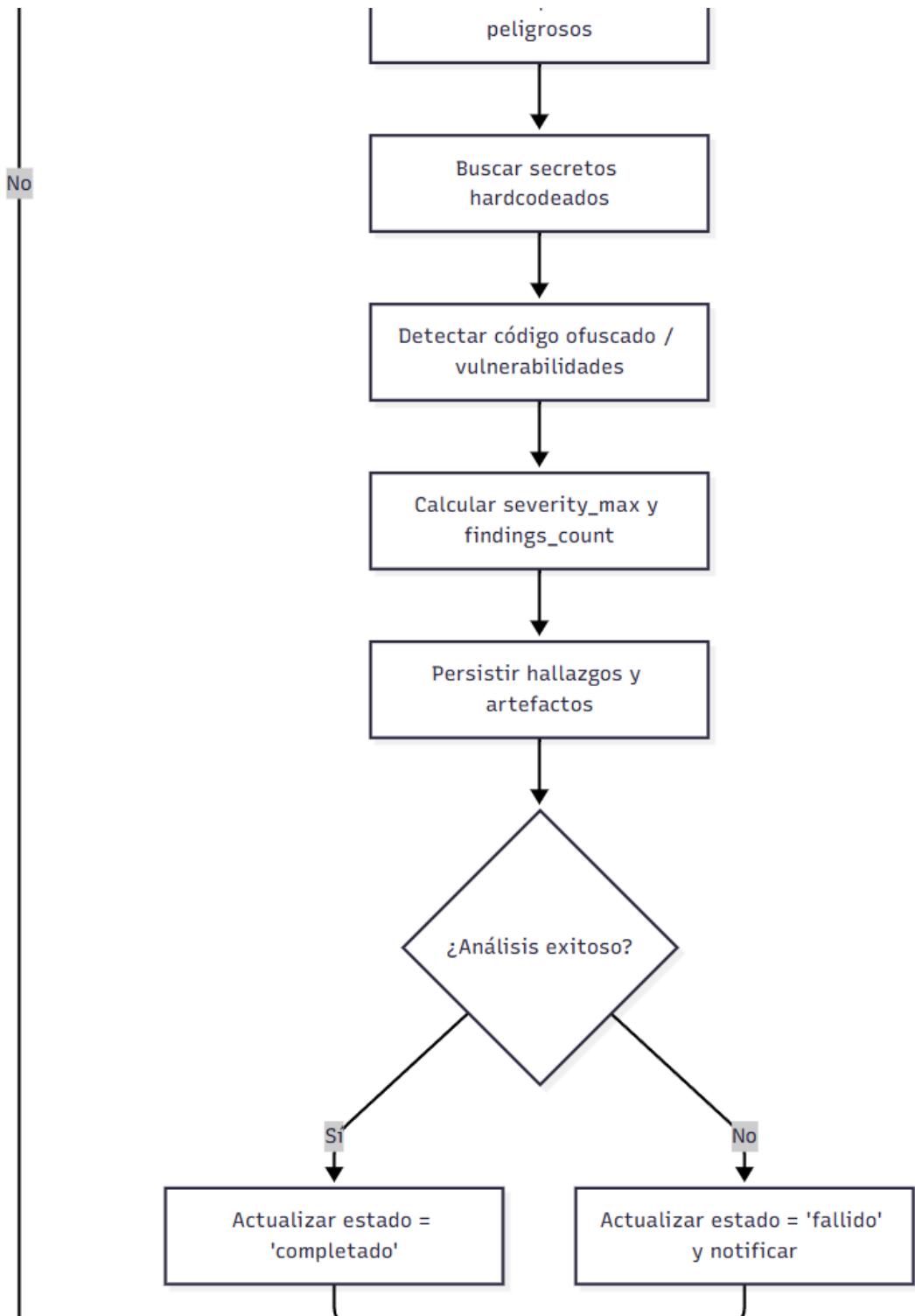


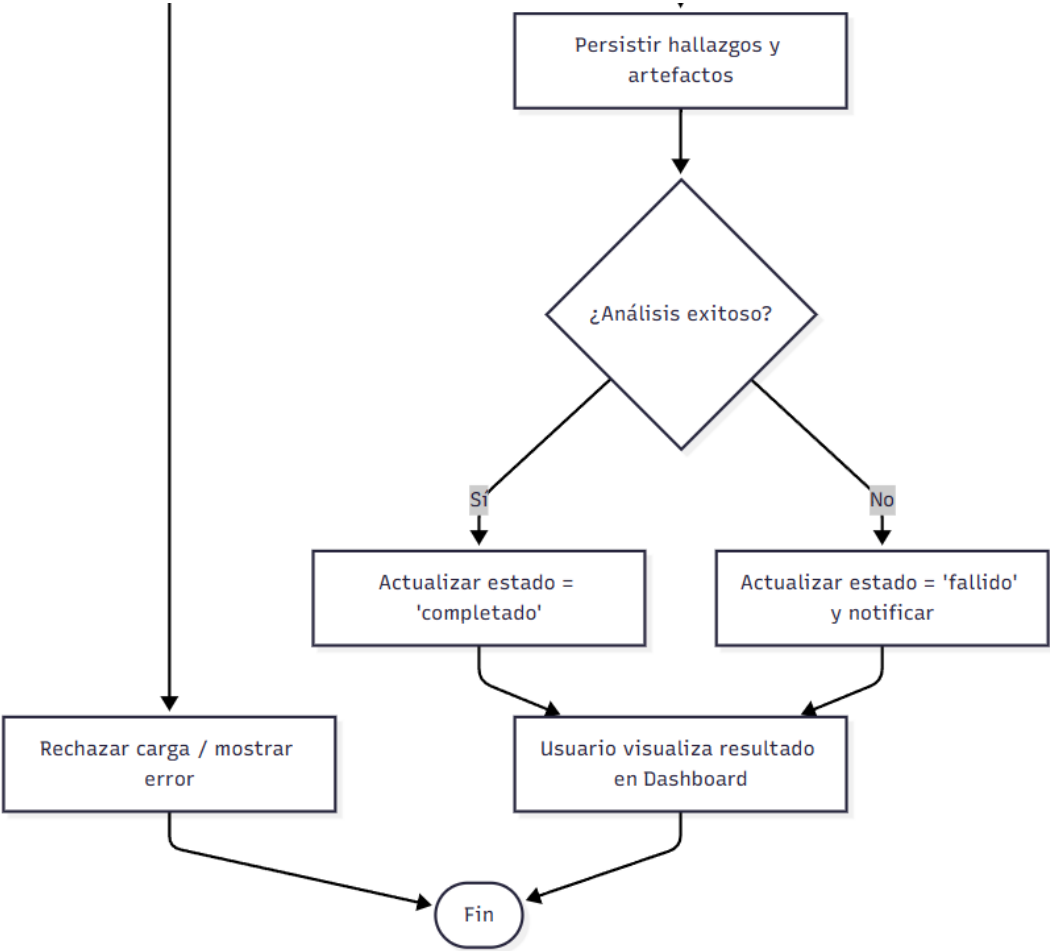
3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)



3.4. Vista de procesos

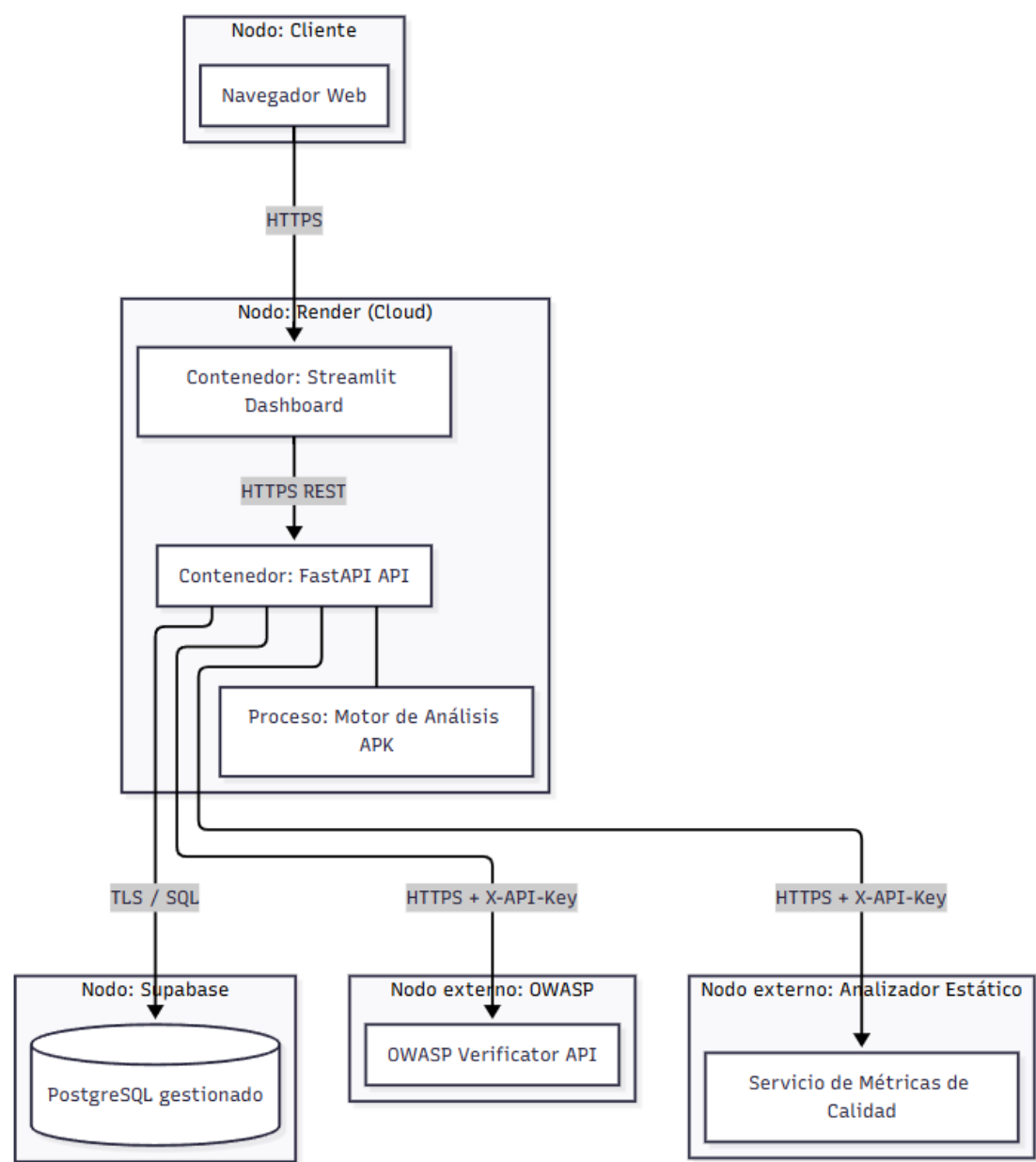
3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)





3.5. Vista de Despliegue (vista física)

3.5.1. Diagrama de despliegue



4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

.

Escenario de Funcionalidad

Escenario AnzenCore: ante una APK subida por el usuario, el sistema debe analizarla completamente (permisos peligrosos, secretos hardcodeados, código ofuscado y vulnerabilidades conocidas) y reflejar correctamente el resultado en el dashboard, garantizando que toda la funcionalidad descrita en los RF-04 a RF-11 del ERS se ejecute sin errores en condiciones normales de operación.

Escenario de Usabilidad

Escenario AnzenCore: todo reporte de análisis (APK, código, URL o repositorio) debe transmitirse cifrado con TLS 1.2 o superior (RNF-01), y el sistema no debe leer, acceder ni almacenar contactos, mensajes, fotos o el contenido binario de la APK del usuario (RNF-02, RN-02), preservando la confidencialidad de la información personal frente a fallos o ataques.

Escenario de confiabilidad

Escenario AnzenCore: todo reporte de análisis (APK, código, URL o repositorio) debe transmitirse cifrado con TLS 1.2 o superior (RNF-01), y el sistema no debe leer, acceder ni almacenar contactos, mensajes, fotos o el contenido binario de la APK del usuario (RNF-02, RN-02),

preservando la confidencialidad de la información personal frente a fallos o ataques.

Escenario de rendimiento

Escenario AnzenCore: el análisis completo de una APK (extracción Dalvik, detección de permisos, secretos y vulnerabilidades) debe completarse en menos de 60 segundos bajo condiciones normales de carga (RNF-03), y la arquitectura debe soportar hasta 10 000 usuarios concurrentes sin degradar el tiempo de respuesta de la API ni del dashboard (RNF-08).

Escenario de mantenibilidad

Escenario AnzenCore: el backend debe seguir estrictamente el patrón MVC en Python (RNF-07), de forma que agregar un nuevo tipo de análisis (por ejemplo, un nuevo escáner de vulnerabilidades) implique únicamente añadir un nuevo módulo en la capa Modelo/Controlador, sin modificar la capa Vista (Streamlit) ni el resto de los analizadores existentes.

Otros Escenarios

Escenario AnzenCore: durante un pico de demanda académica (por ejemplo, varios usuarios subiendo APKs simultáneamente al final de un semestre), la API debe seguir respondiendo dentro de los tiempos esperados (menos de 60 segundos por análisis), gracias al procesamiento

asíncrono del Motor de Análisis APK, que evita bloquear el hilo principal de la API mientras se ejecuta la ingeniería inversa del paquete.